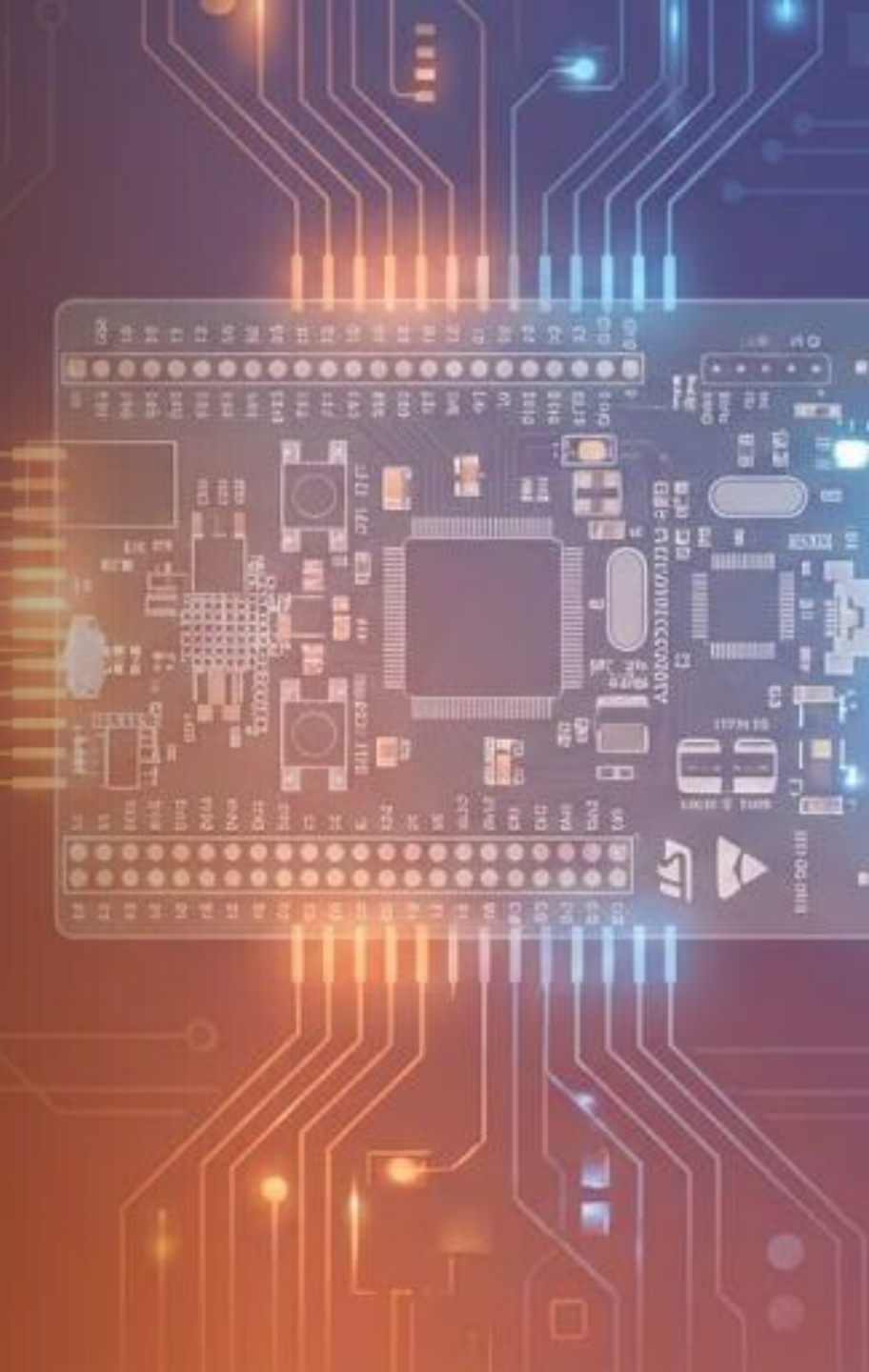
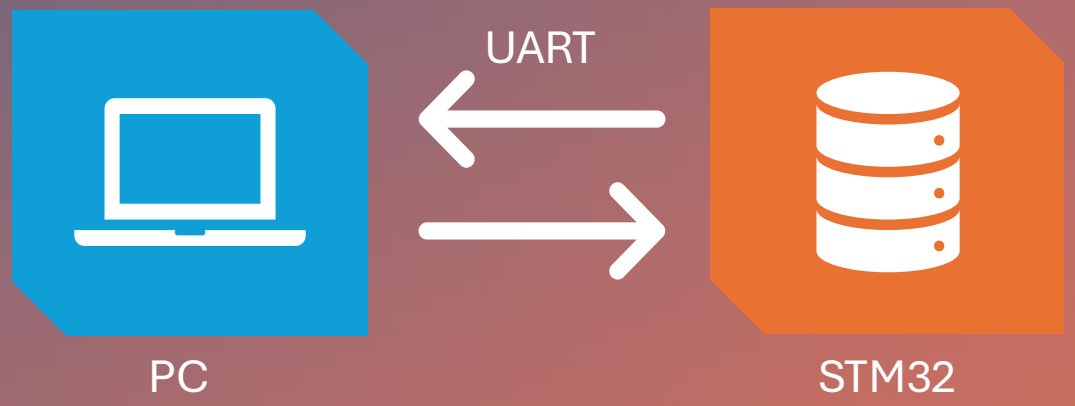
Hardware and software complex for the game ‘Mahjong’

Integration of a PC and STM32 microcontroller via UART



Goal and Project idea



- **The main idea:** Divide the task into 2 independent parts: microcontroller and computer.
- **STM32 (Game logic):** Generates the field, stores its current state and strictly controls the rules (whether a token can be taken, whether the pair matches).
- **PC (Graphics and Control):** Draws a layout on the screen, reads mouse clicks and transmits commands to the microcontroller.
- **Connection:** Data is transmitted via a serial port (UART) at a speed of.

Project structure

HARDWARE: THE BOARD CONNECTS TO A COMPUTER VIA A STANDARD CABLE AND A USB-UART ADAPTER.

PC SOFTWARE (PYTHON): VISUALISATION IS IMPLEMENTED USING THE TKINTER LIBRARY.

STM32 PROGRAMME (C): WRITTEN USING THE HAL LIBRARY. COMMANDS FROM THE PC ARE PROCESSED VIA INTERRUPTS FOR QUICK RESPONSE TO THE PLAYER'S CLICK.

Connection between PC and microcontroller

Data transfer format

Proprietary communication format: Instead of long text (ASCII), we transmit short numbers (bytes). This is much faster and easier for the microcontroller.

Message structure: Only 3 elements: [Command] + [Data] + [Checksum].

Interference protection: Mathematical verification has been added. If the data is damaged in the cable, the system will notice this and automatically send it again.

Memory super-efficiency: Instead of complex 3D coordinates (X, Y, Z), only the sequential number of the chip (from 0 to 49) is transmitted.

Data transfer frame structure

(using the SELECT command as an example)

Example of the SELECT command

Strings: [0x04] [0x12] [0x16]

- 0x04 — Command (SELECT).
- 0x12 — Data (Tile index 18 in hexadecimal system).
- 0x16 — CRC (0x04 XOR 0x12).

Game logic on STM32 (Game Engine)

Field generation: The tiles are shuffled completely randomly.

Spatial check:

- The algorithm checks the Z axis: whether

the tile is covered by another tile above it (4

possible overlap positions).

- The algorithm checks the X axis: whether

the tile is blocked by neighbours on both sides at the same time.

- Validation (cmd_match): Distinguishing between regular suits and special groups (Flowers/Seasons).

Client interface in Python (UI)

- Graphics: Dynamic generation of a 3-level pyramid (5X5, 4X4, 3X3) with shadow simulation.
- Smooth operation: A separate class of UARTHandler class reads data from the board invisibly to the user, so the interface responds to clicks instantly and without lag.
- Fault tolerance: If the cable is disconnected, the player receives a window with a Retry prompt, and the game continues without losing progress.

Challenges and solutions

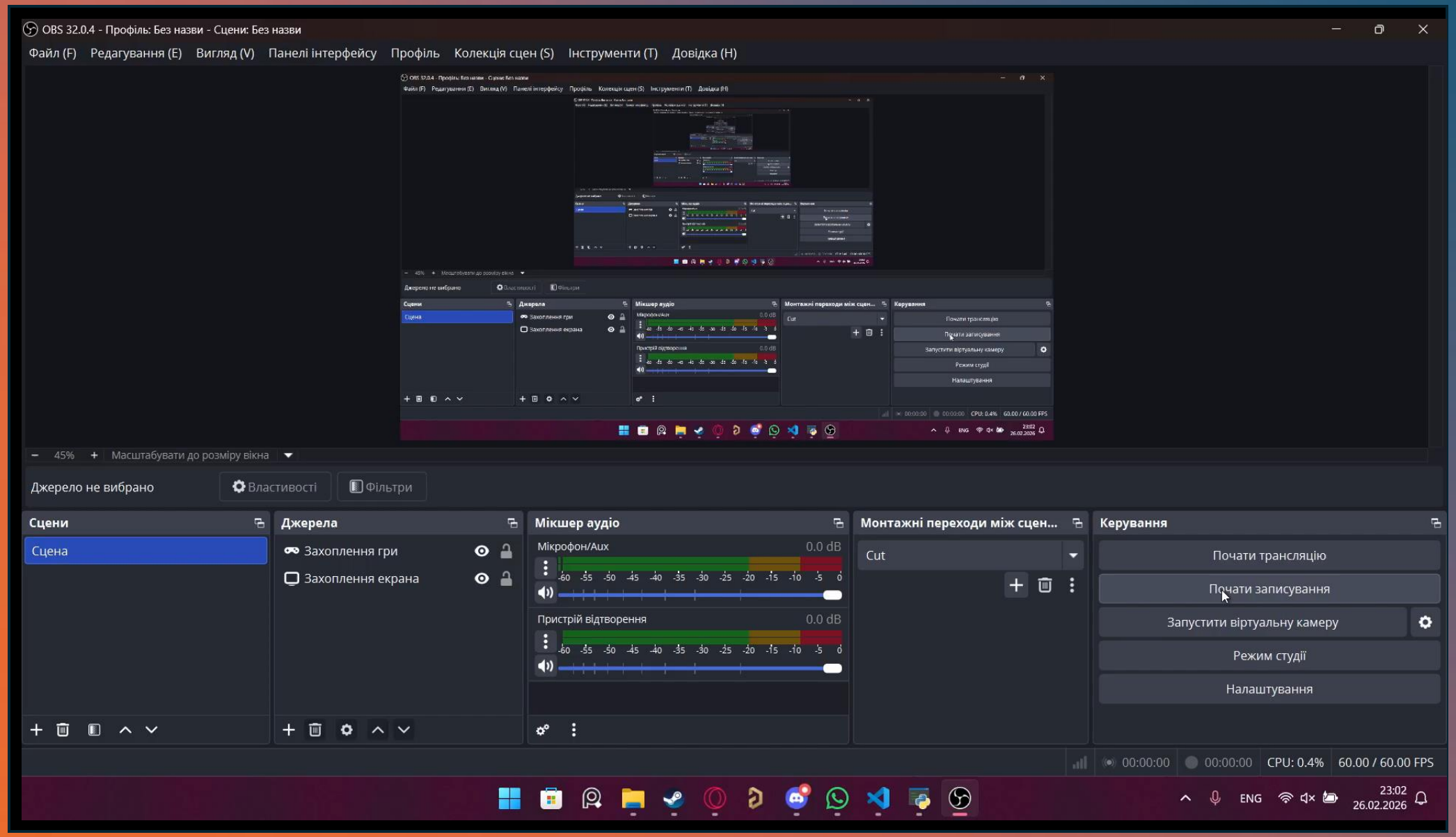
Sudden connection drops

- Disconnecting the cable caused the game to crash on the PC.
- Solution: Development of a timeout mechanism and a reconnection dialogue box without losing game progress.

Data exchange optimisation

- Text commands (ASCII) or transmission of full X, Y, Z coordinates overloaded the microcontroller.
- Solution: Switch to a strict binary format [CMD][DATA][CRC]. Use a single index (0-49) instead of complex spatial coordinates.

Practical implementation: Demonstration of the game process



Plans for the future

- **Statistics and progress:** Implementation of a system for tracking level completion speed and saving player records.
- **Possible addition of new game levels:** Addition of more classic Mahjong shapes.
- **Visuals:** Update of the game's visual style.